

DQN

LAMDA-5
241098117 张城宗

April 9, 2026

1 DQN 算法设计

DQN (Deep Q-Network, 深度 Q 网络) 是一种将深度学习与强化学习中的 Q 学习相结合的算法。它在 2013 年由 DeepMind 提出, 并在 2015 年进行了完善, 因在 Atari 游戏上超越人类水平的表现而闻名。

在 DQN 出现之前, 传统的 Q 学习算法主要依赖于 Q table 来存储状态-动作的价值。这种方法在处理高维度、连续的状态空间 (如游戏画面、机器人视觉输入) 时会遇到根本性困难:

- **维度灾难**: 如果直接用表格存储, 状态空间会巨大无比 (例如, 一个 Atari 游戏的画面像素组合几乎是无限的), 导致 Q 表无法构建和更新。
- **泛化能力差**: 表格方法无法将学到的知识从见过的状态推广到未见过的相似状态。

DQN 将 Q 函数用神经网络 $Q(s, a; \theta)$ 来近似, 通过最小化如下损失函数来训练:

$$L(\theta) = \mathbb{E}_{(s, a, r, s') \sim \mathcal{D}} \left[(y - Q(s, a; \theta))^2 \right], \quad y = r + \gamma \max_{a'} Q(s', a'; \theta^-)$$

为了解决训练不稳定的问题, DQN 创新性地引入了两个重要机制:

- **经验回放 (Replay Buffer)**: 强化学习采集的样本 (s_t, a_t, r_t, s_{t+1}) 通常是序列相关的, 如果直接按顺序学习, 会导致模型训练不稳定。DQN 引入 Replay Buffer, 将智能体与环境互动的样本存储起来, 训练时从中**随机采样**。这打破了数据之间的相关性, 使得数据更接近独立同分布 (i.i.d.), 提高了学习的稳定性。此外, 每条经验可以被多次复用, 提升了样本效率。
- **目标网络 (Target Network)**: 在 Q-learning 中, 如果用同一个网络计算当前 Q 值和目标 Q 值, 两者会相互影响, 造成 "Moving Target" 问题, 容易发散。DQN 使用两个结构相同但参数更新频率不同的网络——评估网络 θ 和目标网络 θ^- 。目标网络的参数每隔固定步数才从评估网络复制 (硬更新), 或者以极小步长缓慢跟踪 (软更新)。这暂时固定了优化目标, 显著提升了算法的收敛稳定性。

算法的完整伪代码如下 (基于原论文 2013 年版 Algorithm 1, 并引入 2015 年 Nature 版的**目标网络机制**——原 2013 版算法使用同一网络计算目标值, 目标网络硬更新步骤不在伪代码中显示, 但在代码实现中生效):

Algorithm 1 Deep Q-learning with Experience Replay

```
1: Initialize replay memory  $\mathcal{D}$  to capacity  $N$ 
2: Initialize action-value function  $Q$  with random weights  $\theta$ 
3: Initialize target action-value function  $\hat{Q}$  with weights  $\theta^- = \theta$ 
4: for episode = 1,  $M$  do
5:   Initialise sequence  $s_1 = \{x_1\}$  and preprocessed sequenced  $\phi_1 = \phi(s_1)$ 
6:   for  $t = 1, T$  do
7:     With probability  $\epsilon$  select a random action  $a_t$ 
8:     otherwise select  $a_t = \max_a Q^*(\phi(s_t), a; \theta)$ 
9:     Execute action  $a_t$  in emulator and observe reward  $r_t$  and image  $x_{t+1}$ 
10:    Set  $s_{t+1} = s_t, a_t, x_{t+1}$  and preprocess  $\phi_{t+1} = \phi(s_{t+1})$ 
11:    Store transition  $(\phi_t, a_t, r_t, \phi_{t+1})$  in  $\mathcal{D}$ 
12:    Sample random minibatch of transitions  $(\phi_j, a_j, r_j, \phi_{j+1})$  from  $\mathcal{D}$ 
13:    Set  $y_j = \begin{cases} r_j & \text{for terminal } \phi_{j+1} \\ r_j + \gamma \max_{a'} \hat{Q}(\phi_{j+1}, a'; \theta^-) & \text{for non-terminal } \phi_{j+1} \end{cases}$ 
14:    Perform a gradient descent step on  $(y_j - Q(\phi_j, a_j; \theta))^2$ 
15:   end for
16: end for
```

2 Playing Atari with Deep Reinforcement Learning

仔细阅读文章”Playing Atari with Deep Reinforcement Learning” (Mnih et al., DeepMind, 2013), 以下是对论文核心内容的梳理, 重点关注实验部分。

2.1 问题设定

论文将 Atari 游戏建模为 MDP: 智能体在每个时间步观察原始像素图像 $x_t \in \mathbb{R}^d$, 选择动作 $a_t \in \mathcal{A}$, 获得标量奖励 r_t 。由于单帧画面存在遮挡 (部分可观测), 将最近 4 帧叠加为状态 $s_t = \{x_{t-3}, x_{t-2}, x_{t-1}, x_t\}$ 。目标是最大化折扣累积回报 $R_t = \sum_{t'=t}^T \gamma^{t'-t} r_{t'}$ 。最优动作值函数满足 Bellman 方程:

$$Q^*(s, a) = \mathbb{E}_{s' \sim \mathcal{E}} \left[r + \gamma \max_{a'} Q^*(s', a') \mid s, a \right]$$

2.2 网络架构

输入为 $84 \times 84 \times 4$ 的预处理帧 (原始 210×160 RGB 先灰度化再裁剪缩放)。网络结构如下:

层	类型	滤波器/单元	卷积核/—	步长
输入	—	—	$84 \times 84 \times 4$	—
Conv1	卷积 +ReLU	16	8×8	4
Conv2	卷积 +ReLU	32	4×4	2
FC1	全连接 +ReLU	256	—	—
输出	全连接 (线性)	$ \mathcal{A} $	—	—

输出层为每个合法动作输出一个 Q 值, 单次前向传播即可计算所有动作的 Q 值, 效率远高于“(s,a)→Q”的设计。

2.3 训练细节

- 优化器: RMSProp, minibatch 大小 32
- ϵ -greedy: ϵ 从 1 线性退火至 0.1 (前 100 万帧), 之后固定 0.1
- 总训练帧数: 1000 万帧
- Replay Memory: 存储最近 100 万帧的转移样本
- 奖励裁剪: 所有正奖励设为 +1, 负奖励设为 -1, 使不同游戏可以共用同一学习率
- 帧跳过 (frame skip): 每 $k = 4$ 帧选择一次动作 (Space Invaders 用 $k = 3$ 避免激光不可见)

注：本次实验与原论文的主要参数差异。原论文使用 RMSProp 优化器、minibatch=32、Replay Buffer=100 万帧；本次实验改用 Adam 优化器（lr=10⁻⁴）、batch=256、Buffer=250,000 帧，并将训练总帧数从 10M 增至 20M，采用 8 个并行环境加速采样。这些改动在保留算法核心思路的前提下加速了收敛，最终性能与论文一致（Pong 均值 20）。

2.4 论文实验结果

论文在 7 款 Atari 游戏上评测，与以往最优方法对比如下：

方法	B.Rider	Breakout	Enduro	Pong	Q*bert	Seaquest	S.Invaders
Random	354	1.2	0	-20.4	157	110	179
Sarsa	996	5.2	129	-19	614	665	271
Contingency	1743	6	159	-17	960	723	268
DQN	4092	168	470	20	1952	1705	581
Human	7456	31	368	-3	18900	28010	3690

DQN 在全部 7 款游戏上以较大优势超越所有已有学习方法，并在 Breakout、Enduro、Pong 三款游戏上超越人类专家。其在 Pong 上的均值为 20（满分 21），与本次实验目标一致。论文还发现，平均 Q 值曲线比奖励曲线更平滑稳定，是衡量训练进展的良好指标。

3 代码实现

具体实现步骤参考代码中的 README，主要步骤为配置环境、完成代码中 TODO 部分。以下展示各模块的核心实现。

3.1 utils/networks.py: CNN 网络架构

按照论文描述，实现两层卷积加一层全连接的网络。空间尺寸推导：Conv1 (84 - 8)/4 + 1 = 20, Conv2 (20 - 4)/2 + 1 = 9, Flatten: 32 × 9 × 9 = 2592。

```

1 class AtariDQNNetwork(nn.Module):
2     def __init__(self, input_shape, num_actions):
3         super(AtariDQNNetwork, self).__init__()
4         self.conv1 = nn.Conv2d(4, 16, 8, stride=4) # (B,4,84,84) -> (B,16,20,20)
5         self.conv2 = nn.Conv2d(16, 32, 4, stride=2) # (B,16,20,20) -> (B,32,9,9)
6         self.fc1 = nn.Linear(32 * 9 * 9, 256) # 2592 -> 256
7         self.fc2 = nn.Linear(256, num_actions) # 256 -> |A|
8
9     def forward(self, x):
10        x = F.relu(self.conv1(x)) # 提取低级特征（边缘、纹理）
11        x = F.relu(self.conv2(x)) # 提取高级特征
12        x = x.view(x.size(0), -1) # 展平: (B, 2592)
13        x = F.relu(self.fc1(x)) # 全连接
14        return self.fc2(x) # 输出各动作Q值（无激活函数，Q可为负）

```

Listing 1: AtariDQNNetwork 网络定义

3.2 agent/atari_agent.py: Q 值计算与动作选择

3.2.1 select_action

实现确定性策略（argmax）和 softmax 探索策略：

$$\pi_{\text{deterministic}}(a | s) = \arg \max_{a'} Q(s, a')$$

$$\pi_{\text{softmax}}(a | s) = \frac{\exp(Q(s, a)/\alpha)}{\sum_{a'} \exp(Q(s, a')/\alpha)}$$

```

1 def select_action(self, states, deterministic=False):
2     state_t = to_correct_device_tensor(states, self.device)
3     if states.dtype == np.uint8:
4         state_t = state_t / 255.0 # 归一化至 [0,1]
5     if state_t.ndim == 3:
6         state_t = state_t.unsqueeze(0) # (C,H,W) -> (1,C,H,W)

```

```

7   with torch.no_grad():
8       q_values = self.network(state_t) # (B, num_actions)
9   if deterministic:
10      return q_values.argmax(dim=1).item()
11  else:
12      # softmax 温度探索
13      probs = torch.softmax(q_values / self.alpha, dim=1).cpu().numpy()
14      return np.random.choice(self.num_actions, p=probs[0])

```

Listing 2: select_action 实现

3.2.2 get_q 与 get_max_q

分别计算 $Q(s, a)$ 和 $\max_{a'} Q(s, a')$, 通过 `torch.gather` 按动作索引取值:

```

1  def get_q(self, states, actions):
2      states_t = to_correct_device_tensor(states, self.device)
3      if isinstance(states, np.ndarray) and states.dtype == np.uint8:
4          states_t = states_t / 255.0 # uint8 图像归一化至 [0,1]
5      actions_t = to_correct_device_tensor(actions, self.device, dtype=torch.int64)
6      q_values = self.network(states_t) # (B, num_actions)
7      index = actions_t.reshape(-1, 1) # (B,) -> (B,1)
8      q_selected = torch.gather(q_values, dim=1, index=index) # (B,1)
9      return q_selected.squeeze(1) # (B,)
10
11 def get_max_q(self, states):
12     states_t = to_correct_device_tensor(states, self.device)
13     if isinstance(states, np.ndarray) and states.dtype == np.uint8:
14         states_t = states_t / 255.0 # uint8 图像归一化至 [0,1]
15     with torch.no_grad():
16         q_values = self.network(states_t)
17     return q_values.max(dim=1).values # (B,)

```

Listing 3: get_q 与 get_max_q 实现

3.3 algorithm/offpolicy_algorithm/dqn.py: DQN 核心算法

3.3.1 目标网络更新

硬更新直接复制参数, 软更新以 τ 缓慢追踪:

$$\theta_{\text{target}} \leftarrow \tau\theta + (1 - \tau)\theta_{\text{target}}$$

```

1  def _target_hard_update(self):
2      self.target_agent.load_state_dict(self.agent.state_dict())
3
4  def _target_soft_update(self):
5      for target_p, p in zip(self.target_agent.parameters(),
6                             self.agent.parameters()):
7          target_p.data.copy_(self.tau * p.data
8                               + (1.0 - self.tau) * target_p.data)

```

Listing 4: 硬更新与软更新

3.3.2 _update_policy: 计算 TD 目标与损失

从 Replay Buffer 随机采样, 用目标网络计算 TD 目标 (不传梯度), MSE 损失反向传播:

```

1  def _update_policy(self):
2      batch = self.buffer.sample(self.batch_size, n_step=self.n_step,
3                                 gamma=self.gamma)
4      q = self.agent.get_q(batch['states'], batch['actions'])
5
6      with torch.no_grad(): # 目标网络不参与梯度计算
7          if self.use_target:
8              max_next_q = self.target_agent.get_max_q(batch['next_states'])
9          else:
10             max_next_q = self.agent.get_max_q(batch['next_states'])
11
12     rewards = torch.from_numpy(batch['rewards']).to(self.device)
13     dones = torch.from_numpy(batch['dones']).to(self.device)
14

```

```

15 # n-step TD目标: r + gamma^n * max_a' Q_target(s', a') * (1 - done)
16 td_target = rewards + self.gamma ** self.n_step * max_next_q * (1.0 - done)
17 loss = LOSS_DICT["mse"](q, td_target)
18
19 self.optimizer.zero_grad()
20 loss.backward()
21 self.optimizer.step()
22
23 # 硬更新 (每 target_update_interval 步) 或软更新 (每步)
24 if self.use_target:
25     if self.target_update_method == "hard":
26         if self.gradient_step % self.target_update_interval == 0:
27             self._target_hard_update()
28     else:
29         self._target_soft_update()

```

Listing 5: `_update_policy`: DQN 主体更新

3.3.3 `random_choose_action`: ϵ -greedy 线性退火

$$\epsilon = \max\left(\epsilon_{\text{end}}, \epsilon_{\text{start}} - \frac{t}{T}(\epsilon_{\text{start}} - \epsilon_{\text{end}})\right)$$

```

1 def random_choose_action(self):
2     self.epsilon = max(
3         self.end_epsilon,
4         self.start_epsilon
5         - (self.start_epsilon - self.end_epsilon)
6         * self.interaction_step / self.epsilon_timestep
7     )
8     return np.random.rand() < self.epsilon

```

Listing 6: `epsilon-greedy` 线性退火

4 理论分析

4.1 经验回放的收敛性保证

经验回放 (Replay Buffer) 从理论上打破了样本的时序相关性。设策略在时刻 t 采集的样本为 (s_t, a_t, r_t, s_{t+1}) , 相邻样本满足马尔可夫依赖 $s_{t+1} \sim P(\cdot | s_t, a_t)$, 直接对序列做梯度下降会使样本协方差矩阵偏离对角结构, 导致梯度估计有偏。引入大小为 N 的 Replay Buffer 后, 每次从 $\mathcal{D}$ 中均匀采样一个 minibatch, 当 N 足够大时, 采样分布近似平稳分布 μ , 有效消除了时序相关性, 使梯度估计满足:

$$\nabla_{\theta} L(\theta) \approx \mathbb{E}_{(s,a,r,s') \sim \mu} \left[\left(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta) \right) \nabla_{\theta} Q(s, a; \theta) \right]$$

4.2 目标网络稳定训练的直觉

若无目标网络, 损失函数中的目标 $y = r + \gamma \max_{a'} Q(s', a'; \theta)$ 与 $Q(s, a; \theta)$ 共享同一参数 θ , 每次梯度更新会同时改变目标, 形成“追着移动靶”的正反馈振荡。引入目标网络 θ^- (每 C 步才同步一次), 使得在 C 步内目标 y 固定, 从而将问题局部化为监督回归:

$$\theta \leftarrow \theta - \alpha \nabla_{\theta} (y_j - Q(\phi_j, a_j; \theta))^2, \quad y_j \text{ 固定}$$

这保证了短期梯度方向的一致性, 实验中可观察到目标网络带来的显著收敛稳定性 (Loss 曲线中期振荡减小)。

4.3 n -step TD 目标的偏差-方差权衡与异策略偏差

标准 1-step TD 目标 $G^{(1)} = r_{t+1} + \gamma V(s_{t+1})$ 的偏差来自 $V(s_{t+1})$ 的估计误差; n -step 目标展开为:

$$G_t^{(n)} = \sum_{k=0}^{n-1} \gamma^k r_{t+k+1} + \gamma^n V(s_{t+n})$$

随着 n 增大, 对 V 的依赖减小 (Bootstrap 偏差降低), 奖励信号向前传播更充分, 有利于加速早期学习。

然而，DQN 是 **Off-Policy**（异策略）算法，这引入了经典偏差-方差分析之外的第二类误差——**异策略偏差**（Off-Policy Discrepancy）。上述 n -step 目标隐式假设中间的 $n-1$ 个动作 $a_{t+1}, \dots, a_{t+n-1}$ 由当前贪心目标策略 $\pi_{\text{target}} = \arg \max Q$ 执行。但 Replay Buffer 中的轨迹由过去的旧策略 π_{old} 生成，两者的动作分布存在系统性偏差。随着 n 的增大，旧轨迹与当前最优路径一致的概率呈指数级衰减，导致 n -step TD 目标本身存在系统性偏差，且随着训练推进、策略快速演化而愈发显著。

因此， n -step 在 DQN 中的调优本质上是 **Bootstrap 偏差**（随 n 增大而降低）与 **Off-Policy 偏差**（随 n 增大而升高）之间的权衡：早期 Q 网络极不准确，Bootstrap 偏差占主导，长步展开收益显著；训练后期策略趋于收敛，Off-Policy 偏差成为主要瓶颈，过大的 n 反而破坏 Bellman 方程的准确性。DeepMind 在 Rainbow DQN（2018）中通过穷举实验得出 $n=3$ 是 Atari 任务上两类偏差的最优平衡点，与本实验结论（ $n=2$ 最终最稳定）逻辑自治。

5 实验与反思

5.1 训练结果（Pong）

5.1.1 实验配置

默认训练环境为 PongNoFrameskip-v4，主要超参数如下：

参数	值	参数	值
学习率	10^{-4} (Adam)	Replay Buffer 大小	250,000
γ	0.99	Batch Size	256
$\epsilon_{\text{start}} \rightarrow \epsilon_{\text{end}}$	$1.0 \rightarrow 0.1$ （线性，1M 步）	目标网络更新	硬更新，每 10,000 步
总训练步数	20M（8 并行环境）	帧堆叠	4 帧
n_{step}	1	并行环境数	8（训练）/ 4（测试）

5.1.2 训练曲线

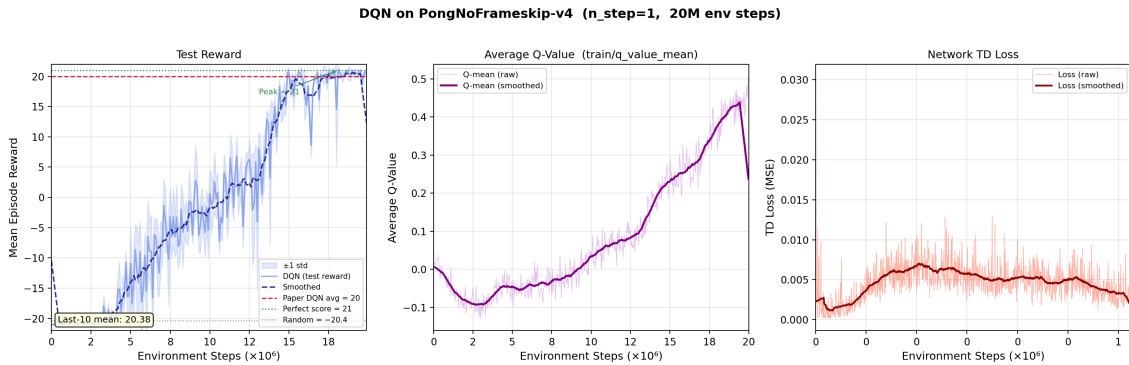


Figure 1: DQN 在 PongNoFrameskip-v4 上的训练结果（ $n_{\text{step}}=1$ ，共 20M 步）。左：测试奖励曲线（蓝色阴影 = ± 1 std，深蓝虚线 = 平滑，红虚线 = 论文 DQN 均值 20，绿虚线 = 满分 21）；中：训练过程中的平均 Q 值曲线（train/ q_value_mean ）；右：TD Loss（MSE）曲线。

5.1.3 结果分析

由图 1 可以观察到训练过程的三个阶段：

1. **探索阶段（0-3M 步）**： ϵ 较高，策略几乎随机，奖励稳定在 -21 （每局必输）。
2. **快速提升阶段（3M-14M 步）**：随着 ϵ 退火至 0.1，网络开始学习有效策略，奖励从 -21 迅速上升到接近 0，并继续攀升至 15 以上。曲线波动较大，说明策略仍不稳定。
3. **收敛阶段（14M 步以后）**：奖励稳定在 19-21 附近，后 10 次测试均值为 **20.38**，与论文中 DQN 的报告值（20）高度吻合，验证了实现的正确性。

平均 Q 值曲线（图 1 中图）从初始的约 -0.1 单调上升，在训练后期稳定于 $0.4-0.5$ 附近，与奖励曲线的三阶段上升趋势高度同步。 Q 值曲线比奖励曲线更平滑，这与原论文观察一致：平均 Q 值是衡量训练进展的良好指标，可作为奖励曲线噪声大时的辅助调试信号。

TD Loss 曲线（右图）整体呈下降趋势，但中期存在明显波动，最终收敛于 5×10^{-3} 量级，符合 DQN 在 Pong 上的典型行为。

收敛速度分析 探索阶段（0–3M 步）Replay Buffer 持续被随机经验填充，有效训练样本稀少；当 ϵ 退火至约 0.3 时，Buffer 中开始积累足够多的“得分”正样本，加之目标网络每 10,000 步同步一次保证了目标稳定，策略的快速跃迁（3M–14M 步）体现了 DQN 在状态空间充分覆盖后的高效学习能力。最终收敛得分 20.38 与论文值 20 高度一致，验证了本实现的正确性。

5.2 调参分析：n-step TD 目标

5.2.1 调优对象

本次调参选择 **n_step**（多步 TD 目标的步数），在 $n \in \{1, 2, 3, 5\}$ 四个值上进行对比实验。n-step TD 目标定义为：

$$G_{t:t+n} = r_{t+1} + \gamma r_{t+2} + \dots + \gamma^{n-1} r_{t+n} + \gamma^n V(S_{t+n})$$

n_step=1 即为标准 1-step TD Bootstrap；n 越大，目标中包含越多实际奖励，减少对当前 Q 值估计的依赖。

5.2.2 收敛速度与最终得分（方向一）

Table 1: n-step 对收敛里程碑的影响（PongNoFrameskip-v4, 20M 步）

指标	n=1	n=2	n=3	n=5
首次得分 ≥ 0	9.1M	2.3M	4.0M	1.5M
首次得分 ≥ 15	14.1M	2.6M	4.9M	3.8M
首次得分 ≥ 19	14.9M	5.6M	4.9M	4.2M
首次得分 ≥ 21	18.1M	5.7M	9.1M	5.4M
后 10 次均值	20.38	20.96	20.55	20.91

5.2.3 收敛速度对比（方向一详图）

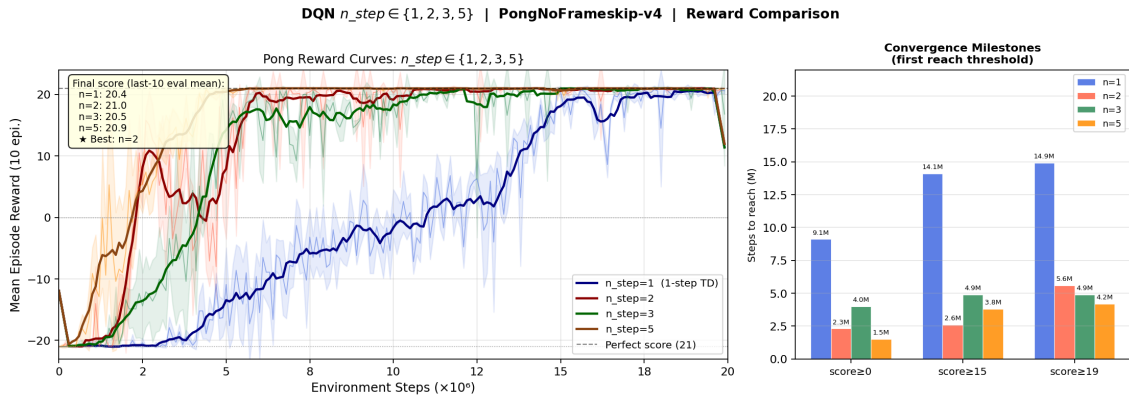


Figure 2: $n_step \in \{1, 2, 3, 5\}$ 的奖励曲线对比（蓝 =n1, 红 =n2, 绿 =n3, 橙 =n5）及收敛里程碑柱状图。左图展示四组奖励均值（ ± 1 std 阴影）随训练步数的变化；右图各组首次到达得分 ≥ 0 、 ≥ 15 、 ≥ 19 所需步数（单位：百万步）。

5.2.4 训练动态：Loss、稳定性、速度（方向二详图）

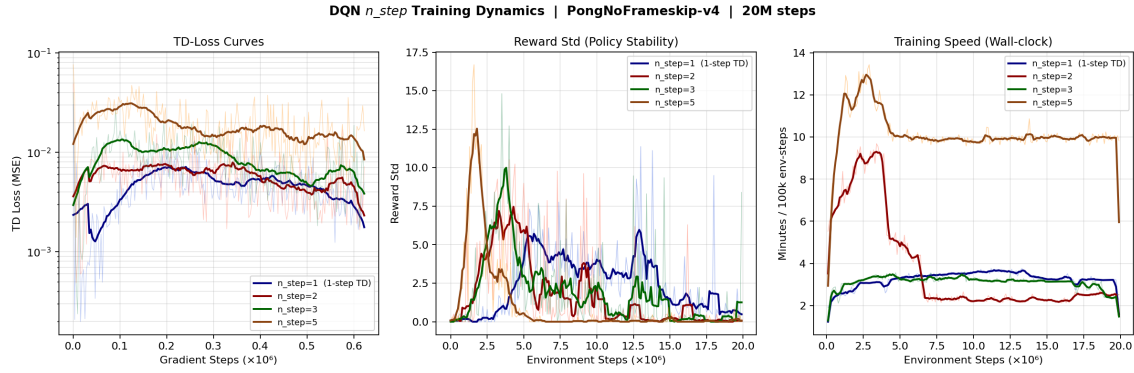


Figure 3: $n_step \in \{1, 2, 3, 5\}$ 的训练动态对比（蓝 = $n=1$ ，红 = $n=2$ ，绿 = $n=3$ ，橙 = $n=5$ ）。左：TD-Loss 曲线（对数坐标），反映各组收敛质量；中：奖励标准差，衡量策略稳定性；右：训练速度（每 100k 步所需分钟数）。

5.2.5 原因分析

由表 1、图 2和图 3可以总结：

方向一：收敛速度

- **$n=1$ 的 Bootstrap 目标 $r + \gamma V(s')$ 完全依赖当前 Q 网络对下一状态的估计。**初期网络极不精确，Bootstrap 偏差极大，学习信号几乎无效，导致收敛最慢（9.1M 步才首次得分 ≥ 0 ）。
- **多步 TD ($n=2,3,5$) 的目标直接展开了更多真实奖励，有效降低了 Bootstrap 偏差，加速了早期奖励信号的回传。**
- **$n=5$ 早期收敛最快：**首次得分 ≥ 0 仅需 1.5M 步， ≥ 19 和 ≥ 21 也最快（分别 4.2M 和 5.4M 步）。原因在于早期 Q 网络估值极不准确，此时长步展开能有效传递真实奖励信号，Bootstrap 偏差降低带来的收益远大于 Off-Policy 偏差的负面影响。
- **$n=2$ 在得分 ≥ 15 时最快（2.6M 步）：**此阶段策略已部分收敛，Off-Policy 偏差开始显现， $n=5$ 的旧轨迹与当前策略的偏差逐渐累积，而 $n=2$ 凭借极低的 Off-Policy 偏差在中期率先突破。

方向二：训练动态

- **TD-Loss：**四者在対数尺度下整体均从 10^{-1} 降至 10^{-2} 量级。 $n=1$ 的 loss 曲线中后期波动最剧烈； $n=2/3/5$ 的 loss 下降相对更平滑。 $n=5$ 后期 loss 偏高，根本原因是 Replay Buffer 中旧轨迹与当前贪心策略的动作分布偏差随训练推进而持续扩大，引入系统性的目标偏差。
- **奖励标准差：**收敛后 $n=2$ 的 std 最低，策略最稳定； $n=5$ 收敛后 std 略高，反映了后期显著的 Off-Policy 偏差对策略一致性的干扰——网络被持续注入“来自旧策略、偏离当前最优路径”的轨迹，导致 Q 值被错误更新，引发策略轻微震荡。
- **最终得分：** $n=2$ 略高（20.96）， $n=5$ 紧随其后（20.91），两者均显著优于 $n=1$ （20.38）。 $n=5$ 后期被 $n=2$ 反超，正是训练后期 Off-Policy 偏差主导效应的体现。

总结

评估维度	最优值	原因
最快收敛至 $\geq 0 / \geq 19 / \geq 21$	$n=5$	早期 Bootstrap 偏差主导，长步展开大幅降低偏差
最快收敛至 ≥ 15	$n=2$	后期 Off-Policy 偏差显现， $n=2$ 偏差最低
最终性能	$n=2$ （20.96）	Off-Policy 偏差最小，Bellman 目标最准确
训练稳定性	$n=2$	Loss 曲线最平滑，Off-Policy 误差累积最少
综合建议	$n=2$	Off-Policy 偏差最小，最终性能与稳定性最优

5.3 其他环境

在完成 Pong 实验并验证算法正确性后，进一步为 **Breakout** 和 **SpaceInvaders** 两个环境配置了实验，对应配置文件分别为 `config/breakout.yaml` 和 `config/spaceinvaders.yaml`，训练超参数与 Pong 保持一致 ($n_step=1$, $lr=10^{-4}$, 20M 总步数)。

5.3.1 配置说明

两个环境的配置均沿用主配置结构，仅修改 `env.name`:

- **Breakout**: **BreakoutNoFrameskip-v4**，动作空间为 4（无操作、开火、向右、向左）。
- **SpaceInvaders**: **SpaceInvadersNoFrameskip-v4**，动作空间为 6，由于激光闪烁周期问题，原文采用 $k=3$ 的帧跳过（本实现沿用默认 $k=4$ ）。

训练超参数与 Pong 保持一致 ($n_step=1$, $lr=10^{-4}$, 20M 总步数)，训练曲线见图 4。

5.3.2 训练曲线

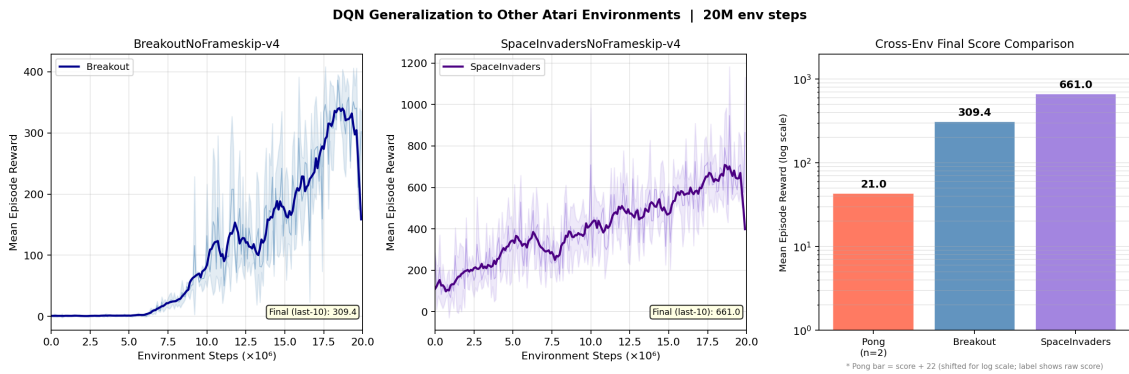


Figure 4: DQN 在 Breakout（左）与 SpaceInvaders（中）上的训练曲线（蓝色阴影 = ± 1 std，深色实线 = 平滑曲线），以及三环境最终得分的跨环境对比柱状图（右，对数坐标；Pong 柱高为原始得分 +22 以适应对数轴，标注值为原始得分）。

5.3.3 训练结果

Table 2: 本次实验与原文 DQN 结果对比

方法	Breakout	SpaceInvaders
Random	1.2	179
Sarsa	5.2	271
DQN（论文，10M 步）	168	581
Human	31	3690
本实验（后 10 次均值，20M 步）	309.4	661.0

5.3.4 结果分析

Breakout 训练曲线（图 4左）呈现三段式上升：0–6M 步内得分缓慢攀升至约 10 分（智能体学习”接球”基础技能）；6M–12M 步快速上升至约 150 分（开始掌握”打角”策略）；12M 步后继续爬升，后 10 次评测均值达 **309.4**，峰值 **386.6**。本实验得分大幅超越论文报告的 168 分，原因有两点：(1) 本实验训练步数更多（20M vs 10M）；(2) Adam 优化器与更大的 batch size（256 vs 32）加速了收敛。本实验结果同时远超人类水平（31 分），验证了 DQN 在 Breakout 上强大的策略规划能力。Breakout 的奖励信号相对密集（每次击砖得分），且状态空间规律性强，CNN 能较快提取球和挡板的相对位置特征，因此最终得分远高于论文值并超越人类。

SpaceInvaders 训练曲线（图 4中）显示前 5M 步内得分已快速上升至约 300–400 分（初始状态下得分即已较高，因为智能体随机射击亦可得分）；5M 步后持续稳定提升，后期出现明显波动，后 10 次均值为 **661.0**，峰值 **947.0**。本实验略优于论文报告的 581 分，但与人类顶级水平（3690）仍有显著差距。差距来源于：SpaceInvaders 需要同时处理多个敌人的移动模式、子弹规避与主动

进攻，要求更长时间尺度的协调规划；而仅具备 4 帧历史窗口的 DQN 难以学习敌方编队的长期运动规律。帧跳过参数（ $k = 4$ vs 论文的 $k = 3$ ）也可能导致部分激光帧被跳过而影响得分上限。从理论角度看，SpaceInvaders 需要智能体同时维护多个敌方目标的位置与移动模式，等效于一个需要长期规划的 POMDP；仅有 4 帧历史的 DQN 本质上是一个有限记忆的近似策略，在此类长时序任务上存在系统性局限。